

INTERNSHIP REPORT

Multimodal Document Parsing, Markdown Integration, and Retrieval-Augmented Generation (RAG) System

Submitted by:

Vaishnav M

Organisation:

Moolya

Duration:

29/07/2026 – 28/08/2026

TABLE OF CONTENTS

- **1. INTRODUCTION**
- 1.1 Background
- 1.2 Problem Statement
- 1.3 Objectives
- **2. TECHNOLOGY EVALUATION AND SELECTION**
- 2.1 Introduction
- 2.2 Experimental Setup
- 2.2.1 Dataset and Input Documents
- 2.2.2 Test Scenarios and Evaluation Criteria
- 2.2.3 Evaluation Methodology
- 2.3 Technology / Component 1 Evaluation (Document Layout Parsers)
- 2.4 Technology / Component 2 Evaluation (Vision-Language Models and OCR)
- 2.4.1 Evaluation of Evaluated Vision-Language Models: Qwen2.5-VL (3B), SmolVLM2 (2.2B), and Qwen3-VL (2B)
- 2.4.2 RapidOCR and VLM Complementary Roles
- 2.5 Technology / Component 3 Evaluation (Audio and Presentation Slide Parsers)
- 2.6 Final Technology Selection
- **3. SYSTEM DESIGN / IMPLEMENTATION OVERVIEW**
- 3.1 Overview
- 3.2 Overall System Architecture
- 3.3 Main Processing Pipeline
- 3.4 Component 1: Pre-Analysis and Smart Page Routing
- 3.5 Component 2: Multi-Engine Document Parsing
- 3.6 Component 3: Multimodal Image Understanding Engine
- 3.7 Component 4: Integration Bridge and Downstream RAG System
- 3.8 Final Deliverables
- 3.9 Technology Stack
- **4. IMPLEMENTATION DETAILS**
- 4.1 Project Structure
- 4.2 Core Module 1: PDF Analysis and Adaptive Page Chunking
- 4.3 Core Module 2: Document Layout Parsers (PyMuPDF, Docling, PPTX, Audio)
- 4.4 Core Module 3: Multimodal Image Filtering, Deduplication, and Analysis
- 4.5 Image Analysis Performance Optimizations (How We Made It Faster)
- 4.6 Core Module 4: Markdown Merging and Document Combination
- 4.7 Core Module 5: Integration Stage (RAG Bridge and Visual Diagram Citations)
- 4.8 Downstream RAG Knowledge System Overview
- 4.9 Unified Command Line Interface (Complete CLI Manual and Commands)

- **5. SYSTEM VERIFICATION AND FUNCTIONAL EVALUATION**
- 5.1 Verification of Document Conversion and Layout Preservation
- 5.2 Verification of Multimodal Image Understanding and Acceleration
- 5.3 Verification of RAG Integration and Visual Citation
- 5.4 Memory Stability and System Reliability
- 5.5 Summary and Conclusion

1. INTRODUCTION

1.1 Background

Every day, organizations work with large volumes of digital documents. These include technical proposals, project specifications, financial reports, presentation slide decks, and audio recordings of meetings.

These documents are rarely just plain text. They are rich with different types of content:

- Formatted paragraphs, headings, and lists
- Complex data tables with rows and columns
- Diagrams, flowcharts, and system architecture drawings
- User interface screenshots and wireframes
- Audio discussions with multiple speakers

To make these documents searchable and interactive, modern software systems use **Retrieval-Augmented Generation (RAG)**. A RAG system takes an organization's documents, processes them into searchable knowledge, and allows team members to ask questions in plain English to get grounded, accurate answers.

However, a RAG system is only as good as the document parser that feeds it. If a parser discards the tables, misses the flowcharts, or throws away the pictures, the system can never answer questions about them.

This project focuses on building an end-to-end **Multimodal Document Parsing, Markdown Integration, and RAG System**:

1. **The Parsing and Multimodal Engine**: Converts complex PDFs, Word documents, PowerPoint slides, and audio recordings into clean, structured Markdown while preserving tables and extracting, cleaning, and describing every informative picture and diagram.
2. **The Integration Bridge**: Seamlessly takes the converted Markdown and image assets and passes them directly to the RAG knowledge system.
3. **The Downstream RAG System**: Enables natural language question-answering and multi-turn chat directly in the terminal, citing supporting text and pointing users to the exact diagram image files that answer their questions.

1.2 Problem Statement

Converting complex digital files and making them searchable presents several major challenges:

4. **Mixed Page Types in Single Documents**:

A single 40-page PDF document has different types of pages. Some pages are plain digital text, others are dense data tables, and others are dominated by architectural diagrams. A simple parser cannot read tables, while a heavy AI parser is far too slow to run on simple text pages.

5. **Computer Memory Crashes**:

Advanced AI document tools (such as deep-learning layout models) use significant computer memory (RAM). When given a large PDF with 30 to 50 pages all at once, these tools frequently crash due to memory exhaustion (`bad_alloc` errors).

6. Loss of Visual Information:

Traditional document converters discard images completely. If a document contains a flowchart showing a six-step approval process, standard converters delete that diagram. Consequently, downstream search systems are completely blind to visual information.

7. Slow Image Analysis and Visual Noise:

Documents contain decorative clutter: thin divider lines, solid color background shapes, tiny bullet icons, and repeated company logos on every header. Sending every tiny icon to a heavy Vision-Language Model slows down the system dramatically and clutters the output.

8. Disconnection Between Document Parsing and Knowledge Retrieval:

Document converters and RAG retrieval pipelines are usually separate, disjointed tools. Users must manually convert files with one program, reformat the outputs with another script, and write custom code to load the text into a retrieval system.

1.3 Objectives

The primary objectives of this internship project are:

9. Design an Automated Page Analyzer:

Inspect each page of a PDF document before converting it. Automatically detect tables and large diagrams, routing simple text pages to a fast parser and complex layout pages to a deep-learning layout parser.

10. Implement Memory-Safe Adaptive Chunking:

Break large documents into safe, manageable execution batches based on the parser type (maximum 5 pages for memory-heavy parsers), preventing computer memory crashes.

11. Evaluate and Select the Best Local Vision-Language Model:

Test and compare modern local Vision-Language Models (`qwen2.5vl:3b`, `smolvlm2-2.2b`, and `qwen3-vl:2b`) to choose the best balance of diagram understanding, reasoning quality, and speed.

12. Maximize Image Analysis Performance:

Implement a full suite of performance optimizations—including heuristic geometric filtering, perceptual hash deduplication, persistent cross-run caching, image resizing, and fast-path OCR labeling—to make image analysis significantly faster.

13. Support Multiple Document Formats:

Support PDF documents, Word documents (`.docx`), PowerPoint slide decks (`.pptx`), and audio recordings (`.mp3`, `.wav`) through a single unified engine.

14. Build the Integration Bridge:

Seamlessly connect the document conversion output with the downstream RAG system, enabling immediate ingestion and visual diagram citations.

15. Provide a Unified Terminal Command Line Interface (CLI):

Allow users to perform end-to-end conversion, RAG ingestion, single-question queries, and interactive chat sessions using simple, standardized terminal commands.

2. TECHNOLOGY EVALUATION AND SELECTION

2.1 Introduction

Before building the system, an extensive technical evaluation was conducted. We evaluated three core technology components:

16. **Document Layout Parsers:** Which tool extracts text and tables with the highest accuracy and best performance?
17. **Vision-Language Models and OCR:** Which vision model and OCR engine best read text and understand relationships inside technical diagrams?
18. **Office and Audio Converters:** Which tools best handle PowerPoint presentation slides and recorded audio meetings?

All technologies were selected with an emphasis on **accuracy**, **table preservation**, **memory stability**, and the ability to run **locally on standard computer hardware** without requiring paid third-party cloud APIs.

2.2 Experimental Setup

2.2.1 Dataset and Input Documents

The evaluation was performed using a diverse corpus of real-world documents:

- **Technical Reports:** Multi-page PDF reports containing dense paragraphs, numbered sections, financial data tables, and embedded system screenshots.
- **Government Tenders:** Formal tender documents containing dense legal clauses, bulleted lists, and complex multi-row tables.
- **Architectural Proposals:** Large documents featuring multi-step system flowcharts, hub-and-spoke network diagrams, and logos on page headers.
- **Presentation Slide Decks:** PowerPoint slides containing slide titles, bullet points, native presentation tables, and embedded graphics.
- **Meeting Audio Recordings:** Spoken technical discussions used to verify speech-to-text transcription.

2.2.2 Test Scenarios and Evaluation Criteria

The system was evaluated against five functional criteria:

19. **Text Fidelity:** Are paragraph headings, text words, and numbered lists preserved accurately without character corruption?
20. **Table Structure Preservation:** Are complex tables converted into clean, aligned Markdown tables without missing rows or collapsed columns?
21. **Visual Diagram Understanding:** Does the vision model extract text inside diagrams and explain the sequential connections between flowchart steps?
22. **Processing Efficiency:** Does image analysis run efficiently without stalling the pipeline on non-informative graphics?
23. **Memory Stability:** Does the system run reliably without memory crashes on documents with 30+ pages?

2.2.3 Evaluation Methodology

Each tool was evaluated through functional testing:

- **Visual Inspection:** Comparing the converted Markdown side-by-side with the original PDF pages.
- **Table Integrity Testing:** Checking whether financial and tabular data retained their row and column alignment.
- **Image Usability Check:** Verifying that extracted PNG images were clear and readable in the assets folder.
- **Vision Model Output Comparison:** Comparing how accurately different VLMs describe complex system flows and architecture diagrams.

2.3 Technology / Component 1 Evaluation (Document Layout Parsers)

1. PyMuPDF (`pymupdf4llm`)

A C-based, lightweight PDF parsing library.

- **Strengths:** Extremely fast; uses very little computer memory; extracts embedded pictures directly.
- **Weaknesses:** Cannot detect complex table borders or cell alignments. Tables are output as jumbled text lines.

2. IBM Docling (`DocumentConverter`)

A deep-learning document understanding framework created by IBM.

- **Strengths:** Outstanding table reconstruction using TableFormer neural networks; converts complex tables into clean Markdown tables; layout-aware reading order.
- **Weaknesses:** Slower processing; heavy memory footprint; prone to memory exhaustion crashes if fed large PDFs without chunking.

Selected Strategy: Adaptive Hybrid Routing

We implemented an automated hybrid approach:

- A fast pre-scanner examines each page.
- Pages containing plain text are routed to **PyMuPDF** for rapid extraction.
- Pages containing tables or large diagrams (>10% page area) are routed to **Docling** for high layout accuracy.

Feature	PyMuPDF Alone	Docling Alone	Selected Hybrid Approach
Speed	Extremely Fast	Slower (Deep Learning)	Fast text + Accurate tables
Table Formatting	Poor (breaks lines)	Excellent (clean tables)	Excellent
Memory Consumption	Minimal (< 100 MB)	High (> 2 GB)	Controlled & Memory-Safe

Crash Risk on Large Files	None	High	None (via Adaptive Chunking)
---------------------------	------	------	------------------------------

2.4 Technology / Component 2 Evaluation (Vision-Language Models and OCR)

To analyze the visual content inside documents (flowcharts, architecture blocks, database schemas, and screenshots), we evaluated three prominent local Vision-Language Models (VLMs) and one dedicated OCR engine.

2.4.1 Evaluation of Evaluated Vision-Language Models

We tested three Vision-Language Models available locally through Ollama:

24. ``qwen2.5vl:3b`` (Alibaba Cloud / Qwen Team):

- **Model Size:** ~3.2 GB
- **Strengths:** High visual reasoning capability. Very good at reading small text and recognizing standard flowchart shapes.
- **Weaknesses:** Noticeably slower inference speed on local consumer hardware. Higher memory footprint, which caused system slowdowns when running alongside layout parsers on systems with limited resources.

25. ``smolvlm2-2.2b`` / ``smolvlm2-2.2b-instruct`` (Hugging Face):

- **Model Size:** ~1.1 GB to 2.5 GB
- **Strengths:** Very lightweight and compact. Loads quickly into memory and executes fast on CPU/GPU.
- **Weaknesses:** Struggles with dense technical flowcharts, branching decisions, and fine entity-relationship labels. In multi-step architecture diagrams, it often missed directional arrows and produced generic descriptions rather than step-by-step technical relationships.

26. ``qwen3-vl:2b`` / ``qwen3-vl:2b-instruct`` (Selected Model):

- **Model Size:** ~1.9 GB
- **Strengths:** The optimal balance for local document analysis. It features strong instruction-following capabilities, reliably adheres to structured output prompts (`## Text Extract` and `## Description`), and traces sequential relationships ([Step 1] --> [Step 2]) with high fidelity. Its 1.9 GB size comfortably fits within local memory limits without competing for resources.
- **Weaknesses:** Occasionally misses faint text in low-contrast graphics (resolved by pairing it with RapidOCR).

Model	Size	Flowchart Tracing	Small Text Accuracy	Speed on Local Hardware	Selection Decision
`qwen2.5vl:3b`	3.2 GB	Very Good	Good	Slower (High resource load)	Evaluated
`smolvlm2-2.2b`	1.1–2.5 GB	Moderate	Fair	Fast	Evaluated
`qwen3-vl:2b`	1.9 GB	High	Good	Fast & Responsive	Selected Primary VLM

2.4.2 RapidOCR and VLM Complementary Roles

While Qwen3-VL understands relationships, it can occasionally miss isolated text tokens. To solve this:

- **RapidOCR (`rapidocr-onnxruntime`)**: A lightweight OCR engine running on CPU via ONNX Runtime. It extracts raw text words in just a few hundredths of a second.

- **Combined Dual Strategy:**

- 27. RapidOCR runs first to extract all visible text tokens.
- 28. Qwen3-VL receives the image along with the OCR tokens to organize the relational sequence.
- 29. If the VLM produces minimal output, RapidOCR tokens automatically populate the Markdown blockquote, ensuring **zero visual text loss**.

2.5 Technology / Component 3 Evaluation (Audio and Presentation Slide Parsers)

1. Audio Transcription: `faster-whisper`

Uses OpenAI's Whisper model optimized with CTranslate2 and 8-bit quantization (**int8**) on CPU.

- **Result:** Transcribes spoken audio smoothly while adding precise time markers ([00:15 - 00:32]).

2. Slide Parsing: `python-pptx`

Directly inspects the XML tree of PowerPoint files.

- **Result:** Extracts slide titles, body bullet points, native PowerPoint tables, presenter notes, and high-resolution embedded graphics.

2.6 Final Technology Selection

Component	Selected Technology	Primary Role in the System
Page Layout Pre-Analysis	PyMuPDF (fitz.get_drawings, get_images)	Inspects page lines and image areas to route pages
Fast Text Parsing	PyMuPDF (pymupdf4llm)	Ultra-fast text extraction for standard pages
Table & Layout Parsing	IBM Docling (DocumentConverter)	Deep-learning table and column structure extraction
Image Noise Filtering	Pillow (PIL.Image)	Eliminates tiny icons, lines, and solid colors
Image Deduplication	imagehash (Perceptual Hashing)	Suppresses repeated header logos across pages
Image OCR	RapidOCR (rapidocr-onnxruntime)	Fast, guaranteed CPU text token extraction
Diagram Understanding	Qwen3-VL:2B-Instruct (via Ollama)	Analyzes flowchart steps and system architectures
Slide Parsing	python-pptx	Extracts slides, tables, and notes from .pptx files
Audio Transcription	faster-whisper (int8 on CPU)	Fast speech-to-text with timestamps
Integration Bridge	Python Subsystem (rag_bridge.py)	Ingests Markdown into RAG & extracts visual citations

3. SYSTEM DESIGN / IMPLEMENTATION OVERVIEW

3.1 Overview

The system is structured as an automated, multi-stage processing pipeline. When a user runs a conversion command:

- 30. Input files are inspected and routed to the proper file pipeline.
- 31. PDF pages are pre-analyzed and grouped into memory-safe execution chunks.
- 32. Text and tables are converted into Markdown, and images are extracted to an assets folder.
- 33. Images are filtered, deduplicated, and analyzed with OCR and a local Vision-Language Model.
- 34. The Markdown text and image descriptions are merged into a unified document (document.md).
- 35. **Integration Stage:** The unified Markdown and asset paths are handed over to the RAG bridge, which ingests the document into the downstream knowledge base.
- 36. **Interactive Query & Chat:** Users can ask questions directly through the terminal. The system retrieves supporting text and cites the exact image files needed to explain visual concepts.

3.2 Overall System Architecture

The document conversion and integration pipeline consists of the following clear layers:

- **Input Ingestion Layer:** Accepts PDF, DOCX, PPTX, and Audio files through the unified command line interface.
- **Pre-Analysis & Chunking Layer:** Scans each PDF page using vector drawing analysis, detecting tables and image ratios to assign pages to PyMuPDF or Docling while grouping pages into memory-safe batches.
- **Conversion & Extraction Layer:** Concurrently extracts text, tables, and raw embedded images using the designated layout engines.
- **Multimodal Visual Intelligence Layer:** Filters out visual noise using geometric rules, deduplicates repeating logos via perceptual hashing, extracts text tokens with RapidOCR, and generates relational summaries using Qwen3-VL.
- **Markdown Assembly Layer:** Injects structured figure tags and blockquote captions into the text and combines all chunks into a unified Markdown document.
- **Integration & RAG Layer:** Hands off the Markdown and assets to the RAG bridge, which creates the searchable knowledge base, resolves visual diagram citations, and serves terminal queries and interactive chat.

3.3 Main Processing Pipeline

The execution workflow follows a clear sequential order:

37. CLI Invocation and Safe Archiving:

The user enters a command in the terminal. If an earlier output directory exists, the system automatically moves it to an archive folder to prevent accidental data loss.

38. Page-by-Page Content Pre-Analysis:

The PDF pre-analyzer checks every page for lines (tables) and picture dimensions. Each page is marked for either PyMuPDF or Docling.

39. Adaptive Page Chunking:

Consecutive pages assigned to the same parser are grouped together. To keep computer memory stable, Docling chunks are capped at 5 pages, while PyMuPDF chunks can include up to 20 pages.

40. Multi-Engine Parsing:

PyMuPDF converts plain text pages rapidly; Docling reconstructs tables and multi-column layouts into Markdown format.

41. Image Filtering and Deduplication:

Extracted images pass through 5 geometric and color filters to remove decorative lines and icons. Perceptual hashing suppresses repeating header logos.

42. Dual OCR and VLM Analysis:

Informative images are processed by RapidOCR for text tokens and by Qwen3-VL for sequential flowchart descriptions. Results are stored in a persistent cache.

43. Markdown Assembly:

The system injects figure numbers, image links, and blockquote descriptions into the Markdown. All chunk files are combined into one unified [document.md](#).

44. Integration and Knowledge Base Ingestion (When `--rag` is passed):

The unified Markdown file is handed over to the RAG bridge, which ingests the content into the downstream knowledge base, enabling instant terminal querying and interactive chat.

3.4 Component 1: Pre-Analysis and Smart Page Routing

The **PDF Analyzer** acts as the traffic controller:

- **Table Detection:** Scans for horizontal and vertical vector lines and grid rectangles. If a page contains 3+ horizontal lines, 3+ vertical lines, or aligned text blocks, it is flagged as containing a table.
- **Large Image Detection:** Calculates the ratio of image area to total page area. If an image covers more than **10% of the page area**, it is flagged as a diagram.
- **Router Assignment:**
 - Table or large image present: Routed to **Docling**.
 - Plain digital text: Routed to **PyMuPDF**.

The **Adaptive Chunker** groups pages according to parser limits and uses PyMuPDF to create lightweight temporary PDF files for processing.

3.5 Component 2: Multi-Engine Document Parsing

45. **PyMuPDF Parser:**

- Converts standard text pages into clean Markdown in fractions of a second.
- Extracts embedded pictures and standardizes image links into comment placeholders.

46. **Docling Parser:**

- Uses deep-learning TableFormer models to detect table cells, rows, and columns.
- Outputs clean Markdown tables with rows and columns.
- Extracts pictures at high resolution.

47. **PowerPoint Parser:**

- Reads slides sequentially, turning PowerPoint native tables into Markdown tables.
- Captures slide titles, bullet points, speaker notes, and embedded pictures.

48. **Audio Transcriber:**

- Uses faster-whisper on CPU (8-bit quantization) to transcribe spoken words into text.
- Formats speech with clear timestamps (e.g., [00:15 - 00:32] Speaker text...).

3.6 Component 3: Multimodal Image Understanding Engine

The image pipeline consists of four layers:

- 49. **5-Rule Geometric Filter:** Rejects images smaller than 60x60 pixels, thinner than 30 pixels, smaller than 3,000 square pixels, banner aspect ratios wider than 6:1, and single solid-color boxes.
- 50. **Perceptual Hash Deduplication:** Creates a visual fingerprint. Repeating header logos across multiple pages are analyzed once and suppressed on subsequent pages.
- 51. **Fast OCR Text Extraction:** Extracts all visible words and numbers from diagrams and screenshots.
- 52. **Relational VLM Analysis:** Examines flowchart arrows and component interactions and writes a 2 to 4 sentence technical summary.

3.7 Component 4: Integration Bridge and Downstream RAG System

The **RAG Bridge** connects the document conversion engine directly with the downstream retrieval system:

- 53. **Automated Ingestion:** Immediately accepts the generated document.md file without manual file copying.
- 54. **Clean-Slate Default:** Clears prior index data so queries only reference the current upload (with an optional --append flag to keep earlier documents).
- 55. **Visual Citation Engine:** Scans retrieved text passages for image tags. It resolves the local file path on disk, extracts the nearby diagram description, and displays a dedicated **RELEVANT DIAGRAMS & IMAGES** section in the terminal.
- 56. **Interactive Chat & Query Runner:** Provides both single-question answers and multi-turn terminal conversation with full conversation history memory.

3.8 Final Deliverables

The engine produces the following deliverables:

- 57. ``document.md``: A unified Markdown document containing text, Markdown tables, and structured figure blocks including Figure numbering, asset links, text extracts, and conceptual descriptions.
- 58. ``output/assets/``: High-resolution PNG files for every extracted image.

3.9 Technology Stack

Layer	Technology	Version	Purpose
Language	Python	3.11	Primary development language
PDF Text Parsing	PyMuPDF (fitz, pymupdf4llm)	1.24+	Fast text and picture extraction
PDF Table Parsing	IBM Docling & Docling-Core	2.0+	Deep-learning table and layout extraction
Slide Parsing	python-pptx	1.0+	PowerPoint slides, tables, and notes
Audio Processing	faster-whisper	1.0+	Fast speech-to-text with CPU int8 quantization
Image OCR	RapidOCR (rapidocr-onnxruntime)	1.3+	Fast CPU-based text token extraction
Vision-Language Model	Qwen3-VL:2B-Instruct	2B	Flowchart, diagram, and relation understanding
Model Serving	Ollama	0.4+	Local execution of AI models
Image Processing	Pillow (PIL)	10.0+	Image resizing, filtering, and conversion
Visual Hashing	imagehash	4.3+	Perceptual hashing for logo deduplication
Integration Bridge	Python Subsystem (rag_bridge.py)	—	Handoff to RAG and visual diagram citations

4. IMPLEMENTATION DETAILS

4.1 Project Structure

The project code is organized into clean, dedicated modules inside `PDFtoMarkdown_V3/` and `qdrant_vectordb/`:

- ``main.py``: Root entry point handling command-line arguments and routing to conversion or RAG commands.
- ``requirements.txt``: Complete list of Python libraries and dependencies.
- ``image_analysis_cache.json``: Persistent JSON cache storing perceptual hashes, OCR tokens, and VLM descriptions.
- ``PDFtoMarkdown_V3/``: Contains the document conversion engine:
 - `modules/pdf_analyzer.py`: Inspects vector drawings and image sizes to assign parsers.
 - `modules/adaptive_chunker.py`: Groups pages into parser-aware chunks and splits PDFs.
 - `modules/pymupdf_parser.py`: Fast text extraction engine.
 - `modules/docling_parser.py`: Deep-learning layout and table parser.
 - `modules/pptx_parser.py`: PowerPoint slide, table, and notes extractor.
 - `modules/audio_transcriber.py`: Speech-to-text transcription engine.
 - `modules/image_filter.py`: Five-rule geometric image filter.
 - `modules/image_analyzer.py`: Dual RapidOCR and Qwen3-VL analysis engine.
 - `modules/markdown_merge.py`: Merges image blockquotes and captions into Markdown.
 - `modules/markdown_combiner.py`: Joins chunk files into the final document.
 - `modules/rag_bridge.py`: Integration bridge that hands off content to RAG and handles visual citations.
- `pipelines/`: Contains dedicated execution pipelines for Hybrid PDF, Docling-only PDF, PPTX, and Audio.
- ``output/``: The final output folder containing `document.md` and the `assets/` subfolder.

4.2 Core Module 1: PDF Analysis and Adaptive Page Chunking

1. Page Analysis (``pdf_analyzer.py``)

The page analyzer opens the PDF document and scans each page before any parsing takes place:

- **Table Detection Algorithm**: It inspects vector drawings on the page. It counts horizontal line segments, vertical line segments, and rectangle bounding boxes. If a page has 3 or more horizontal lines, 3 or more vertical lines, or 10 or more cell rectangles, it is classified as a table page. It also checks for borderless tables by analyzing aligned text column coordinates.
- **Large Image Detection**: It computes the pixel area of every embedded image. If an image occupies more than 10% of the total page area, it is identified as an important visual diagram or flowchart.
- **Routing Decision**: Pages with tables or large diagrams are routed to Docling; all remaining text pages are routed to PyMuPDF.

2. Adaptive Chunker (``adaptive_chunker.py``)

To prevent computer memory overload:

- It scans the list of page assignments and groups consecutive pages assigned to the same parser into execution batches.
- For Docling, batches are capped at a maximum of 5 pages to keep memory usage low.
- For PyMuPDF, batches can contain up to 20 pages because text extraction requires minimal memory.
- For each batch, it generates a temporary PDF file containing only those pages using PyMuPDF's page insertion method.

4.3 Core Module 2: Document Layout Parsers

1. PyMuPDF Integration (``pymupdf_parser.py``)

Converts plain text pages into clean Markdown in fractions of a second. Embedded image tags are standardized into comment placeholders, and non-essential background artifacts are removed.

2. Docling Integration (``docling_parser.py``)

Runs IBM Docling with deep-learning layout models. Table cells, column boundaries, and multi-row alignments are detected and exported directly as GitHub-flavored Markdown tables. Images are extracted at double scale (2x) for high resolution.

3. PowerPoint Parsing (``pptx_parser.py``)

Reads PowerPoint presentations slide by slide. Native presentation table shapes are parsed cell by cell into Markdown tables. Slide titles are converted into Markdown headings, and embedded graphics are saved to the assets folder.

4. Audio Transcription (``audio_transcriber.py``)

Transcribes spoken audio files using **faster-whisper** on CPU with 8-bit quantization. Speech segments are tagged with start and end timestamps so readers can trace the timeline of the recording.

4.4 Core Module 3: Multimodal Image Filtering, Deduplication, and Analysis

The image analysis engine processes images through three coordinated stages:

1. Five-Rule Geometric Filter (``image_filter.py``)

Discards uninformative visual elements before running any AI model:

- **Rule 1 (Tiny Graphics):** Skips images where both width and height are under 60 pixels.
- **Rule 2 (Thin Lines):** Skips images where either width or height is under 30 pixels (such as page dividers).
- **Rule 3 (Small Area):** Skips images with a total pixel area under 3,000 pixels.
- **Rule 4 (Extreme Aspect Ratio):** Skips images with an aspect ratio greater than 6.0 (such as horizontal banner strips).
- **Rule 5 (Solid Color):** Skips single-color background blocks.

2. Perceptual Hash Deduplication (``imagehash``)

Calculates a visual hash for every image. If the exact same logo appears on multiple pages, it is analyzed only once and suppressed on subsequent pages.

3. Dual-Engine Extraction

- **RapidOCR:** Runs on CPU to extract all detected text tokens.
- **Qwen3-VL (Ollama):** Structures the text and explains the diagram workflow into distinct sections.

4.5 Image Analysis Performance Optimizations (How We Made It Faster)

Running a Vision-Language Model on every image extracted from a document can be computationally intensive. To make image analysis significantly faster, we implemented eight specific engineering optimizations:

59. Pre-Filtering Visual Noise (The 5-Rule Filter):

By applying lightweight geometric checks in Python memory before calling any AI model, decorative artifacts (lines, dots, solid backgrounds) are skipped instantly. This eliminates 40% to 50% of unnecessary vision model calls.

60. Perceptual Hash Deduplication:

A 64-bit visual hash is calculated for each image. Repeated header logos and company emblems are identified immediately. The logo is processed once, and duplicate model calls on subsequent pages are suppressed.

61. Persistent Cross-Run Caching:

Analysis results are saved in a persistent JSON cache keyed by perceptual hash. When a previously analyzed image is encountered in any document, its description is returned in 0.00 seconds.

62. Image Downscaling and Normalization:

High-resolution graphics are automatically downscaled to a maximum dimension of 1024x1024 pixels before being passed to Qwen3-VL. This preserves text legibility while significantly cutting down processing time.

63. Fast-Path Labeling for Simple Badges:

When RapidOCR detects only 1 or 2 short words (such as a simple button or badge), the analyzer formats the label directly in Python memory without calling the vision model, completing in 0.01 seconds.

64. Fast CPU-Based RapidOCR:

RapidOCR runs on CPU using multithreaded ONNX Runtime. It extracts raw text tokens in fractions of a second and provides them directly to the vision model prompt, allowing the vision model to focus on conceptual relationships rather than character detection.

65. Zero-Temperature Deterministic Sampling and Multithreading:

The vision model is configured with zero temperature, repeat penalty controls, and 8 CPU processing threads. This eliminates conversational rambling and maximizes local hardware utilization.

66. The Optional `--no-analysis` Flag:

When speed is the top priority and only text and tables are needed, users can supply the `--no-analysis` flag. This completely bypasses the vision model and OCR steps, extracting text and tables while linking raw image files directly.

4.6 Core Module 4: Markdown Merging and Document Combination

1. *Caption Formatter* (`markdown_merge.py`)

Replaces image placeholders with:

- A numbered heading (### Figure X)
- An image link pointing to the local file (assets/image.png)
- Blockquote sections containing the structured text extract and the conceptual description.

2. *Document Combination* (`markdown_combiner.py`)

Collects all chunk files in sequential order, joins them together, and writes the complete output to `output/document.md`.

4.7 Core Module 5: Integration Stage (RAG Bridge and Visual Diagram Citations)

The integration module (`rag_bridge.py`) serves as the bridge between document conversion and knowledge retrieval:

- **Automated Ingestion:** It accepts the list of converted Markdown files and hands them off directly to the downstream knowledge base. By default, it clears earlier index data to provide a clean slate for the current upload, while supporting an `--append` option to retain earlier documents.
- **Visual Diagram Citation:** When a user asks a question, the system searches the knowledge base for relevant text passages. If a retrieved passage contains an image tag, the visual citation engine resolves the relative image path to the local disk file in `output/assets/`, verifies that the image exists, and extracts the nearby description. The system then displays a dedicated **RELEVANT DIAGRAMS & IMAGES** section in the terminal, showing the exact image filename and path so users can view the diagram directly.

4.8 Downstream RAG Knowledge System Overview

The downstream RAG system acts as the interactive knowledge consumer of the converted documents:

67. **Knowledge Ingestion:** Accepts the structured Markdown documents and image assets from the integration bridge, storing the text and layout structures in a local searchable index.
68. **Context Retrieval:** When a question is asked, the system searches the stored knowledge base to find the exact paragraphs, table data, and diagram extracts that address the query.
69. **Visual Diagram Citations:** If a retrieved passage references a diagram or flowchart, the system looks up the corresponding image file in `output/assets/` and cites the local file path directly in the terminal output.
70. **Interactive Terminal Chat:** Enables team members to converse with the document corpus across multiple questions, maintaining full conversation memory.

4.9 Unified Command Line Interface (Complete CLI Manual and Commands)

All operations are performed through the root `main.py` script.

1. Conversion Commands (``convert``)

To convert a single PDF document to Markdown:

```
python main.py convert PDFtoMarkdown_V3/input/test1.pdf --out output/
```

To run fast conversion skipping image analysis (runs quickly without AI vision):

```
python main.py convert PDFtoMarkdown_V3/input/test1.pdf --out output/ --no-analysis
```

To convert an entire folder of mixed files:

```
python main.py convert PDFtoMarkdown_V3/input/ --out output/
```

To convert presentation slides:

```
python main.py convert presentation.pptx --out output/
```

To convert an audio recording with a small whisper model:

```
python main.py convert meeting.mp3 --out output/ --whisper-model small
```

2. End-to-End Conversion + RAG Integration Commands

To convert a document, ingest into RAG, and start interactive chat immediately:

```
python main.py convert PDFtoMarkdown_V3/input/test1.pdf --out output/ --rag --chat
```

To convert a document, ingest into RAG, and ask a single question immediately:

```
python main.py convert PDFtoMarkdown_V3/input/test1.pdf --out output/ --rag --query "What is the project budget?"
```

To run fast conversion without image analysis, followed by immediate RAG chat:

```
python main.py convert PDFtoMarkdown_V3/input/test1.pdf --out output/ --no-analysis --rag --chat
```

To append to the existing knowledge base instead of replacing it:

```
python main.py convert PDFtoMarkdown_V3/input/test2.pdf --out output/ --rag --append --query "List all deliverables"
```

3. Standalone RAG Management Commands (``rag``)

To ingest an existing Markdown file or folder into the knowledge base:

```
python main.py rag --ingest output/
```

To ask a question against already-indexed documents:

```
python main.py rag --query "Explain the recruitment matching workflow"
```

To start an interactive multi-turn terminal chat session:

```
python main.py rag --chat
```


To inspect database status (vector count, collection name, indexed files):

```
python main.py rag --status
```

To clear the entire knowledge base (wipes existing index data):

```
python main.py rag --clear
```

4. Complete CLI Flags and Options Reference

Command	Flag	Type	Default	Description
convert	inputs	Positional	Required	One or more file paths or folder paths to process
convert	--out	Option	Required	Directory where generated Markdown and assets are saved
convert	--pipeline	Choice	hybrid	Processing pipeline: hybrid (adaptive) or docling (all pages)
convert	--no-analysis	Flag	False	Skips VLM and OCR image analysis
convert	--no-summary	Flag	False	Disables summary generation for audio files
convert	--whisper-model	Choice	base	Whisper model size: tiny, base, small, or medium
convert	--rag	Flag	False	Hands off converted Markdown to the RAG pipeline immediately
convert	--append	Flag	False	Appends to existing index (default: clears index for current upload)
convert	--query	String	None	Asks a question immediately after conversion and ingestion

convert	--chat	Flag	False	Launches interactive terminal chat immediately after conversion
rag	--query	String	None	Asks a single question against the indexed documents
rag	--chat	Flag	False	Launches an interactive terminal chat session
rag	--status	Flag	False	Displays collection statistics, vector counts, and indexed files
rag	--clear	Flag	False	Wipes all index data
rag	--ingest	Option	None	Ingests one or more existing Markdown files into the database
rag	--rag-pipeline	Choice	hybrid	Retrieval mode: hybrid, fast, or adaptive

5. SYSTEM VERIFICATION AND FUNCTIONAL EVALUATION

5.1 Verification of Document Conversion and Layout Preservation

The document conversion engine was functionally verified across diverse documents to confirm that text, headings, and data tables were accurately preserved:

71. Heading and Paragraph Structure:

- Section numbers (such as 1.1 and 2.3.1) were properly mapped to standard Markdown headings.
- Bullet lists and numbered sequences retained their indentation and hierarchy.
- Text reading order on multi-column pages was preserved without overlapping or out-of-order paragraphs.

72. Table Structure Preservation:

- Pages identified as containing tables were successfully routed to Docling.
- Complex multi-row and multi-column tables were converted into standard Markdown tables.
- Cell alignments, header rows, and numeric values matched the source PDF documents without column shifting or missed rows.

5.2 Verification of Multimodal Image Understanding and Acceleration

The image filtering and analysis pipeline was tested to confirm that visual diagrams were properly processed while decorative noise was rejected:

73. Noise Filtering Verification:

- The 5-rule heuristic filter successfully identified and discarded thin divider lines, header rules, and spacer graphics.
- Small bullet icons and decorative background color blocks were rejected, preventing unnecessary AI model execution.

74. Logo Deduplication Verification:

- On documents featuring repeating company logos across page headers, the perceptual hashing system recognized the identical visual fingerprint.
- The logo was processed on its first occurrence and suppressed on subsequent pages, preventing visual clutter in the generated Markdown.

75. OCR and Diagram Workflow Verification:

- RapidOCR reliably extracted text tokens and labels from flowcharts, architecture diagrams, and user interface screenshots.
- Qwen3-VL accurately traced sequential flowchart connections and produced clear technical summaries.
- In instances where diagrams had low contrast, the RapidOCR fallback ensured that raw text tokens were retained in the blockquote extract, preventing data loss.

76. Acceleration Impact:

- Using caching, image downscaling, noise filtering, and fast-path labeling significantly reduced image processing overhead, making multimodal document conversion smooth and practical for everyday use.

5.3 Verification of RAG Integration and Visual Citation

The integration bridge between the document conversion engine and the downstream RAG knowledge base was tested through end-to-end command execution:

77. Automated Handoff and Indexing:

- Running conversion with the `--rag` flag successfully created the output Markdown and assets, immediately handing them off to the RAG bridge.
- The document was indexed into the knowledge base without requiring manual file copying or custom scripts.
- By default, the index was refreshed to ensure queries referenced only the current upload, while the `--append` flag successfully retained earlier documents when requested.

78. Visual Diagram Citation Verification:

- Questions concerning system workflows and architectures retrieved relevant text chunks containing image references.
- The system resolved the relative image path to the local disk file in `output/assets/`, verified its presence, and printed the file path in a dedicated **RELEVANT DIAGRAMS & IMAGES** section in the terminal.
- The system accurately referenced both the written context and the specific diagram filename in its response.

5.4 Memory Stability and System Reliability

Memory safety and error handling were verified during long-running conversions:

79. Elimination of Memory Crashes:

- Testing confirmed that unchunked deep-learning parsers frequently caused memory exhaustion (`std::bad_alloc`) on large multi-page documents.
- The Adaptive Chunker resolved this issue by restricting memory-heavy parsing to small batches (maximum 5 pages per chunk).
- Across all document tests, computer RAM usage remained stable, and the system completed all tasks with zero crashes.

80. Safe Output Archiving:

- Repeated conversion runs verified that previous output folders were atomically moved to an archive directory before new outputs were generated, ensuring that earlier work was never overwritten or lost.

5.5 Summary and Conclusion

The developed **Multimodal Document Parsing, Markdown Integration, and RAG System** successfully achieves all project goals:

81. **Intelligent and Adaptive:** Combines the speed of PyMuPDF with the table reconstruction capability of Docling through automated page pre-analysis.
82. **Multimodal Visual Understanding:** Extracts, filters, and describes diagrams, flowcharts, and technical screenshots using RapidOCR and Qwen3-VL, selected through extensive evaluation of modern local VLMs.
83. **High Performance and Speed:** Implements a comprehensive suite of optimizations (heuristic filtering, perceptual hashing, persistent caching, downscaling, and fast labeling) to accelerate image analysis.
84. **Seamless RAG Integration:** Hands off converted Markdown and image assets to the downstream RAG knowledge system via `rag_bridge.py`.
85. **Visual Diagram Citations:** Accurately detects and cites relevant image files in the terminal when answering questions about visual content.
86. **Memory-Safe and Reliable:** Uses adaptive chunking to prevent memory overload on large documents.
87. **Unified Terminal CLI:** Provides simple, standardized commands to convert documents, ingest knowledge, and run interactive chat sessions.

End of Report